

Introductory Lectures on Optimization

Introduction

Yin Zhang
zhangyin98@zju.edu.cn

College of Computer Science, Zhejiang University

Outline

- 1 Interplay of Optimization and Machine Learning
- 2 General Formulation of The Problem
- 3 Performance of Numerical Methods
- 4 Reference

Part I

Interplay of Optimization and Machine Learning

The Machine Learning (Supervised) Paradigm

Consider the following **spaces**: a space of *examples* $\mathcal{X}$, a space of *labels* $\mathcal{Y}$, and a space of *hypothesis* $\mathcal{H}$ that contains functions mapping $\mathcal{X}$ to $\mathcal{Y}$. Also consider a *loss* function $L: \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ and a *probability measure* P over the space $\mathcal{X} \times \mathcal{Y}$.

Definition 1.1

For any hypothesis $h \in \mathcal{H}$ we define the risk of h with respect to L and P to be:

$$\mathcal{L}_P(h) = \mathbb{E}_P(L(h(x), y)).$$

The general problem of machine learning can be then cast as finding the hypothesis $h \in \mathcal{H}$ that solves the following optimization problems:

$$\min_{h \in \mathcal{H}} \mathcal{L}_P(h).$$

*We here follow the narrative in (Munoz, 2014)

Empirical Risk Minimization

The practical way is to solve an *empirical risk minimization* problem. That is

$$\min_{h \in \mathcal{H}} \sum_{i=1}^n L(h(x_i), y_i) = \min_{\theta} \sum_{i=1}^n L(h_{\theta}(x_i), y_i),$$

since we do not know the P exactly. That is

$$\theta_* = \operatorname{argmin}_{\theta} \sum_{i=1}^n L(h_{\theta}(x_i), y_i).$$

- 1 Classification: the output variable takes **class labels**. Especially, for binary classification, $y_i \in \{1, -1\}$.
- 2 Regression: the output variable takes **continuous values**. That is $y_i \in \mathbb{R}$.

Classification: The 0~1 Loss

By using the 0~1 *loss*, we can rewrite the objective function as:

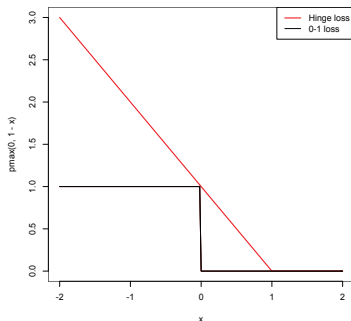
$$\sum_{i=1}^n \mathbf{1}_{\text{sign}(h(x_i)) \neq y_i} = \sum_{i=1}^n \mathbf{1}_{h(x_i)y_i < 0}$$

where $\mathbf{1}_z$ is just the indicator function (Heaviside step function, or the unit step function), that is

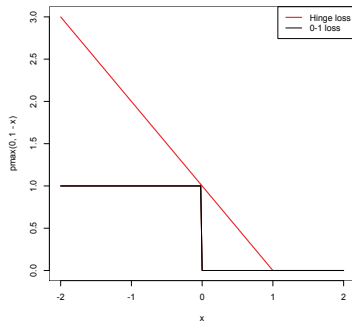
$$\mathbf{1}_z = \begin{cases} 1, & z \text{ is true,} \\ 0, & \text{otherwise.} \end{cases}$$

Classification: The Hinge Loss

In fact, this problem is **NP-hard** so it is pointless to try to minimize this function. To deal with this problem, we can introduce some *surrogate* like the *hinge loss* or *margin loss* given by $\phi(y, y') = \max(0, 1 - y'y)$. This function is an upper bound to the 0~1 loss as shown in the following figure. Coarsely speaking, it is much more easy to solve (We will explain it later).



Classification: The Hinge Loss



It turns out that (Bartlett et al., 2006)

$$\min_{h \in \mathcal{H}} \mathbb{E}[L(h(x), y)] = \min_{h \in \mathcal{H}} \mathbb{E}[\phi(h(x), y)]$$

Classification: The Hinge Loss

Now suppose the hypotheses class is given by linear function

$$h_{\theta}(x) = w^{\top} x + b$$

for $\theta \triangleq (w^{\top}, b)^{\top} \in \mathbb{R}^{n+1}$, thus the classification problem becomes:

$$\min_w \sum_{i=1}^n \max(0, 1 - y_i (x_i^{\top} w + b)).$$

By adding some slack variables, we have

$$\begin{aligned} & \min_w \sum_{i=1}^n \zeta_i \\ \text{s.t. } & y_i (x_i^{\top} w + b) + \zeta_i \geq 1, \\ & \zeta_i \geq 0. \end{aligned}$$

Classification: The Hinge Loss

To control the complexity (make the Rademacher complexity small), we bound the norm of the vector w . So we constrain $\|w\|_2^2 \leq \Lambda$ and obtain

$$\begin{aligned} & \min_w \sum_{i=1}^n \zeta_i \\ \text{s.t. } & y_i(x_i^\top w + b) + \zeta_i \geq 1, \\ & \zeta_i \geq 0, \\ & \|w\|_2^2 \leq \Lambda. \end{aligned}$$

Classification: The Hinge Loss

By introducing a Lagrange multiplier $\lambda > 0$ for the last constrain, we have

$$\begin{aligned} \min_w \quad & \sum_{i=1}^n \zeta_i + \lambda \|w\|_2^2 - \lambda A \\ \text{s.t.} \quad & y_i(x_i^\top w + b) + \zeta_i \geq 1, \\ & \zeta_i \geq 0. \end{aligned}$$

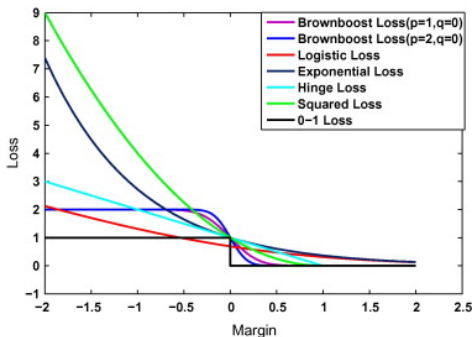
In practice, λ is tuned using cross-validation on a held-out data set. Thus, we commonly assume that λ is a constant. Then we obtain the SVM problem:

$$\begin{aligned} \min_w \quad & \sum_{i=1}^n \zeta_i + \lambda \|w\|_2^2 \\ \text{s.t.} \quad & y_i(x_i^\top w + b) + \zeta_i \geq 1, \\ & \zeta_i \geq 0. \end{aligned}$$

Classification: Other Loss Functions

Other common loss functions for classification are list as follows.

- 1 Square loss: $\phi_s(y, y') = (1 - yy')^2$.
- 2 Logistic loss: $\phi_l(y, y') = \frac{1}{\ln 2} \ln(1 + e^{-yy'})$.
- 3 Cross entropy loss: $\phi_c(y, y') = -t \ln(y') - (1 - t) \ln(1 - y')$ where $t = (1 + y)/2$.



Maximum Likelihood Estimation View

Let us say, we have a likelihood function $P(X, Y|\theta)$. Then the MLE for θ , the parameter we want to infer, is

$$\begin{aligned}\theta_{MLE} &= \operatorname{argmax}_{\theta} P(X, Y|\theta) = \operatorname{argmax}_{\theta} \log P(X, Y|\theta) \\ &= \operatorname{argmax}_{\theta} \log \prod_i P(x_i, y_i|\theta) \\ &= \operatorname{argmax}_{\theta} \sum_{i=1}^n \log P(x_i, y_i|\theta)\end{aligned}$$

For case that $P(x_i, y_i|\theta) \sim \exp -L(h_{\theta}(x_i), y_i)$, we have

$$\theta_{MLE} = \operatorname{argmin}_{\theta} \sum_{i=1}^n L(h_{\theta}(x_i), y_i).$$

Maximum A Posteriori Estimation View

If we replace the likelihood in the MLE formula above with the posterior, we get:

$$\begin{aligned}\theta_{MAP} &= \operatorname{argmax}_{\theta} P(X, Y|\theta) P(\theta) = \operatorname{argmax}_{\theta} \log \{P(\theta) P(X, Y|\theta)\} \\ &= \operatorname{argmax}_{\theta} \log \left\{ P(\theta) \prod_i P(x_i, y_i|\theta) \right\} \\ &= \operatorname{argmax}_{\theta} \left\{ \log P(\theta) + \sum_{i=1}^n \log P(x_i, y_i|\theta) \right\}\end{aligned}$$

For case that $P(x_i, y_i|\theta) \sim \exp -L(h_{\theta}(x_i), y_i)$ and $P(\theta) \sim \exp -\lambda \|\theta\|_2^2$, we have

$$\theta_{MAP} = \operatorname{argmin}_{\theta} \left\{ \sum_{i=1}^n L(h_{\theta}(x_i), y_i) + \lambda \|\theta\|_2^2 \right\}.$$

Some Remarks

We remark here:

- 1 Hypothesis: Not only shallow model, deep learning ...
- 2 Variational Inference: Minimize KL divergence.
- 3 Reinforcement Learning: Maximize a long-term reward.
- 4 Optimization vs. Sampling.

Part II

General Formulation of The Problem

General Formulation of The Problem

令 x 是一个 n 维实向量:

$$x = (x^{(1)}, \dots, x^{(n)})^\top \in \mathbb{R}^n,$$

S 是 $\mathbb{R}^n$ 一个子集, 且 $f_0(x), \dots, f_m(x)$ 是 x 的一些实值函数。我们处理以下一般最小化问题的不同变种:

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S, \end{aligned} \tag{1}$$

其中符号 $\leq$ 可以是 $\leq, \geq$, 或者 $=$ 。

* 可以用 $\leq$ 替代 $\geq$ 和 $=$, 所以后文均使用 $\leq$ 。

General formulation of the problem

$$\begin{array}{ll}\min & f_0(x) \\ \text{s.t.} & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S,\end{array}$$

我们称 $f_0(x)$ 是我们的问题的 **目标** (objective) 函数, 向量函数

$$f(x) = (f_1(x), \dots, f_m(x))^T$$

称为 **函数约束** (functional constraints) 向量, 集合 S 称为 **基本可行集** (basic feasible set), 并且集合

$$Q = \{x \in S \mid f_j(x) \leq 0, j = 1 \dots m\}$$

称为问题 (3) 的 **可行集** (feasible set)。只考虑最小化问题仅仅是一个习惯。

Natural Classification

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S, \\ & Q = \{x \in S \mid f_j(x) \leq 0, j = 1 \dots m\} \end{aligned}$$

对于最小化问题的类型 (types), 存在一个自然的分类:

- 有约束问题 (constrained problems): $Q \subset \mathbb{R}^n$ 。
- 无约束问题 (unconstrained problems): $Q \equiv \mathbb{R}^n$ 。
- 光滑问题 (smooth problems): 所有 $f_j(x)$ 是可微的。
- 非光滑问题 (nonsmooth problems): 存在不可微的分量 $f_k(x)$ 。

Natural Classification

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S, \end{aligned}$$

- **线性约束问题** (linearly constrained problems): 所有泛函约束是线性的。线性约束问题可以继续分为:
 - 线性优化问题: 如果 $f_0(x)$ 也是线性的, 那么式 (3) 是一个**线性优化问题**。
 - 二次优化问题: 如果 $f_0(x)$ 是二次的, 那么式 (3) 是一个**二次优化问题**。
 - 二次约束的二次问题: 如果所有 f_j 是二次的, 那么式 (3) 是一个**二次约束的二次问题** (quadratically constrained quadratic problem)。

Classification based on the properties of feasible set

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S, \\ & Q = \{x \in S \mid f_j(x) \leq 0, j = 1 \dots m\} \end{aligned}$$

基于可行集属性的分类法。

- 问题 (3) 称为**可行的** (feasible), 如果 $Q \neq \emptyset$ 。
- 问题 (3) 称为**严格可行的** (strictly feasible), 如果存在 $x \in Q$, 对于所有不等式约束满足 $f_j(x) < 0$ (或者 > 0), 对于所有等式约束满足 $f_j(x) = 0$ (**Sater condition**)。

Different Types of Solutions

$$\begin{array}{ll}\min & f_0(x) \\ \text{s.t.} & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S,\end{array}$$

最后,区分式 (3) 的解的不同类型:

- x^* 称为式 (3) 的优化**全局解** (global solution), 如果对于所有的 $x \in Q$, 有

$$f_0(x^*) \leq f_0(x).$$

在这种情况下, $f_0(x^*)$ 称为问题的(全局)**最优值** (optimal value)。

- x^* 称为式 (3) 的**局部解** (local solution), 如果对于所有的 $x \in \text{NBR}(x^*, \delta) \cap Q$, 有

$$f_0(x^*) \leq f_0(x).$$

Examples Demonstrating The Origin of The Problem

Example 2.1 (Example 1.1.1)

令 $x^{(1)}, \dots, x^{(n)}$ 是我们的设计变量 (design variables)。那么我们可以固定一些我们决策的泛函特征 (characteristics): $f_0(x), \dots, f_m(x)$ 。固定最重要的特征, $f_0(x)$, 作为我们的目标 (objective)。对于其他的, 我们加上限定: $a_j \leq f_j(x) \leq b_j$ 。因此, 我们得到了如下问题:

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & a_j \leq f_j(x) \leq b_j, j = 1 \dots m, \\ & x \in S, \end{aligned}$$

其中 S 表示了结构化 (structural) 约束, 例如非负性, 或者某些变量的有界性。

Examples Demonstrating The Origin of The Problem

Example 2.2 (Example 1.1.2)

让我们的初始问题形如：

$$\text{寻找 } x \in \mathbb{R}^n \text{ 满足 } f_j(x) = a_j, j = 1 \dots m. \quad (2)$$

那么我们可以考虑问题：

$$\min_x \sum_{j=1}^m (f_j(x) - a_j)^2,$$

也许还要有一些附加的关于 x 的约束。如果后面这个问题的优化值是零，我们说初始问题 (2) 有解。

注意，问题 (2) 几乎是普遍存在的 (universal)。

Examples Demonstrating The Origin of The Problem

Example 2.3 (Example 1.1.3)

有时候我们的决策变量 $x^{(1)}, \dots, x^{(n)}$ 必须是整数 (integer)。这个要求可以用下面的约束来描述：

$$\sin(\pi x^{(i)}) = 0, \quad i = 1 \dots n.$$

因此, 我们也可以处理整数优化 (integer optimization) 问题：

$$\begin{aligned} & \min f_0(x) \\ \text{s.t.: } & a_j \leq f_j(x) \leq b_j, \quad j = 1 \dots m, \\ & x \in S, \\ & \sin(\pi x^{(i)}) = 0, \quad i = 1 \dots n. \end{aligned}$$

Optimism or Pessimism

非线性优化的先驱者们的乐观主义：

非线性优化是非常重要的和有前途的应用理论。它覆盖了几乎所有的运筹学和数值分析的需求。(包括现在的大量机器学习兴趣)

现实主义者的悲观：

总体上看, 优化问题是无解的 (unsolvable)。

然而, 科学没有乐观和悲观, 只有客观。

Part III

Performance of Numerical Methods

A Problem and A Class of Problem

$$\begin{aligned} & \min f_0(x) \\ \text{s.t. } & f_j(x) \leq 0, j = 1 \dots m, \\ & x \in S, \end{aligned} \tag{3}$$

考虑一个求解问题 (3) 方法: 对任何输入, 返回 $x^* = 0$ 。

- 对于解确实是 0 的方法, 这个方法的性能是无法被超越的。
- 显然, 这个方法对所有的问题都无效, 除了 (except) 那些解确实在原点的问题。

因此, 我们无法对于一个特定问题 $\mathcal{P}$ 讨论最好的方法, 但是我们可以对一类 (class) 问题 $\mathcal{F} \ni \mathcal{P}$ 来讨论。的确, 通常某个数值方法被发展出来求解许多不同的问题, 这些问题有相似的特征。

因此, 一个方法 $\mathcal{M}$ 在整个类 $\mathcal{F}$ 上的性能 (performance) 是它的效率的一个自然特征。

A Problem and A Class of Problem

考虑 $\mathcal{M}$ 在一个类 $\mathcal{F}$ 上的性能。我们应该假定 $\mathcal{M}$ 没有关于一个特定的问题 $\mathcal{P} \in \mathcal{F}$ 的完全 (complete) 信息。

- 模型:

问题 $\mathcal{P}$ 的一个已知 (known) 的“部分”, 称为问题的模型 (model)。用 Σ 来表示模型。通常, 模型包含了问题形式, 泛函分量的类型描述, 等等。

- Oracle:

为了识别问题 $\mathcal{P}$ (并且求解它), 方法应该能够搜集关于 $\mathcal{P}$ 的特定的信息。为了方便, 搜集数据的过程被描述为一个 oracle。一个 oracle $\mathcal{O}$ 就是一个单位, 它回答方法连续的询问。方法 $\mathcal{M}$ 试图通过搜集和处理回答, 来求解问题 $\mathcal{P}$ 。

通常, 每个问题可以被不同的模型来描述。此外, 对于每个问题, 可以发展不同类型的 oracles。

A Problem and A Class of Problem

固定 Σ 和 $\mathcal{O}$ 。这样,就很自然地定义了 $\mathcal{M}$ 在 $(\Sigma, \mathcal{O})$ 上的性能,即在**最差的** (worst)问题 $\mathcal{P}_w$ 上的性能。这个问题 $\mathcal{P}_w$ 来自于 $(\Sigma, \mathcal{O})$ 。

进一步,什么是 $\mathcal{M}$ 在 $\mathcal{P}$ 上的**性能** (performance)? 让我们从最直觉的定义开始:

$\mathcal{M}$ 在 $\mathcal{P}$ 上的性能是方法 $\mathcal{M}$ **求解问题** $\mathcal{P}$ 所需的**计算努力** (computational efforts) 的总量。

什么是:求解问题?

求解问题就是要找到一个 $\mathcal{P}$ 的准确度为 $\epsilon > 0$ 的**逼近解**。

General Iterative Scheme

General Iterative Scheme

- 输入: 初始点 x_0 和准确度 $\epsilon > 0$
- 初始化: 设置 $k = 0$, 初始信息集 $I_{-1} = \emptyset$ 。

Main Loop:

- 1 在 x_k 调用 oracle $\mathcal{O}$ 。
- 2 更新信息集: $I_k = I_{k-1} \cup (x_k, \mathcal{O}(x_k))$ 。
- 3 对 I_k 运用方法 $\mathcal{M}$ 的规则, 形成点 x_{k+1} 。
- 4 检查停止条件 $\mathcal{T}_\epsilon$ 。如果满足条件, 则形成输出 $\bar{x}$ 。否则设置 $k := k + 1$ 且转到步骤1

(4)

Complexity

在方案 (4) 中, 可以很容易地找到两个最昂贵的步骤。第一个是步骤 1, 调用 oracle, 第二个是步骤 3, 形成下一个测试点。因此, 针对方法 $\mathcal{M}$, 可以引入两个关于问题 $\mathcal{P}$ 的复杂性的测量:

解析复杂性 (analytical complexity): 求解问题 $\mathcal{P}$, 达到准确率 ϵ , 所要调用 oracle 的数量。

算术复杂性 (arithmetical complexity): 求解问题 $\mathcal{P}$, 达到准确率 ϵ , 所须要的算术操作的总数量(包括 oracle 和方法的工作)。

对于一个应用于 $\mathcal{P}$ 的特定方法 $\mathcal{M}$, 算术复杂度可以很容易地从解析复杂度和 oracle 的复杂度得到。

* 在本课程中, 对某些问题类, 我们主要讨论解析复杂度的界。

Oracle

对于 oracle, 有一个标准的假设, 让我们能得到优化问题解析复杂度的大部分结果。这个假设称为 **局部黑盒概念** (local black box concept), 如下所示:

局部黑盒

- 1 对于数值方法, 仅有的可用信息是 oracle 的回答。
- 2 这个 oracle 是局部的: 距离测试点 x 很远的小的问题变化, 不会改变在 x 的回答。

Oracle

标准形式 (3) 称为优化问题的一个**泛函模型**(functional model)。通常, 对于这样的模型, 标准的假定和函数分量的光滑性相关的。根据光滑性的程度, 我们可以使用不同类型的 oracle:

- **零阶 oracle** (zero-order oracle):
返回 $f(x)$ 的值。
- **一阶 oracle** (first-order oracle):
返回 $f(x)$ 和梯度 $\nabla f(x)$ 的值。
- **二阶 oracle** (second-order oracle):
返回 $f(x)$, 梯度 $\nabla f(x)$, 和 Hessian $\nabla^2 f(x)$ 的值。

References I

- Peter L Bartlett, Michael I Jordan, and Jon D McAuliffe. Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101(473):138–156, 2006.
- Aharon Ben-Tal and Arkadiaei Semenovich Nemirovskiaei. Lectures on modern convex optimization: analysis, algorithms, and engineering applications. 2001.
- Stephen Boyd, Stephen P Boyd, and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- Andres Munoz. Machine learning and optimization. URL: https://www.cims.nyu.edu/~munoz/files/ml_optimization.pdf [accessed 2016-03-02][WebCite Cache ID 6fiLfZvnG], 2014.
- Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*, volume 87. Springer Science & Business Media, 2003.
- Yurii Nesterov. *Lectures on convex optimization*, volume 137. Springer, 2018.

Thank You!